Web Systems Development

What a Framework Gets you: Laravel

RPI

Week 7 readings

Optional book

- Matt Stauffer, Laravel: Up & Running, 3rd ed.
  - Through the RPI library; sign in with RPI credentials
  - Chapter 1, Why Laravel?: the first two sections
  - Chapter 3, Routing and Controllers
  - Chapter 4, Blade Templating
  - Skip chapter 2. The book covers Laravel up to 11; we use 13.

Reference: Laravel 13 docs

- Request lifecycle
- Directory structure
- Routing
- Blade templates
- Controllers
- Requests
- Service container (drill 9)
- HTTP tests (for Friday)

Nothing is required this week. When a book and the docs disagree, the docs win.

TODAY'S ROUTE

Follow one request from hand-built PHP to Laravel

01 **Begin with code you already know**
Week 3's index.php → inversion of control → the request lifecycle
SLIDES 3–5

02 **Trace the request through Laravel**
Front controller, routing, MVC, Blade, layouts, injection and middleware
SLIDES 6–12

03 **See what standardization buys — and costs**
Convention over configuration and the framework bargain
SLIDES 13–14

04 **Build with the pieces**
Routes + Blade reference, Docker workflow, nine drills and next steps
SLIDES 15–18

Keep one question in view: what did index.php do by hand, and what does Laravel now do for you?



Frontend

Focuses on layout, animations, content organization, navigation, graphics.

Programming languages:
JavaScript, HTML, CSS



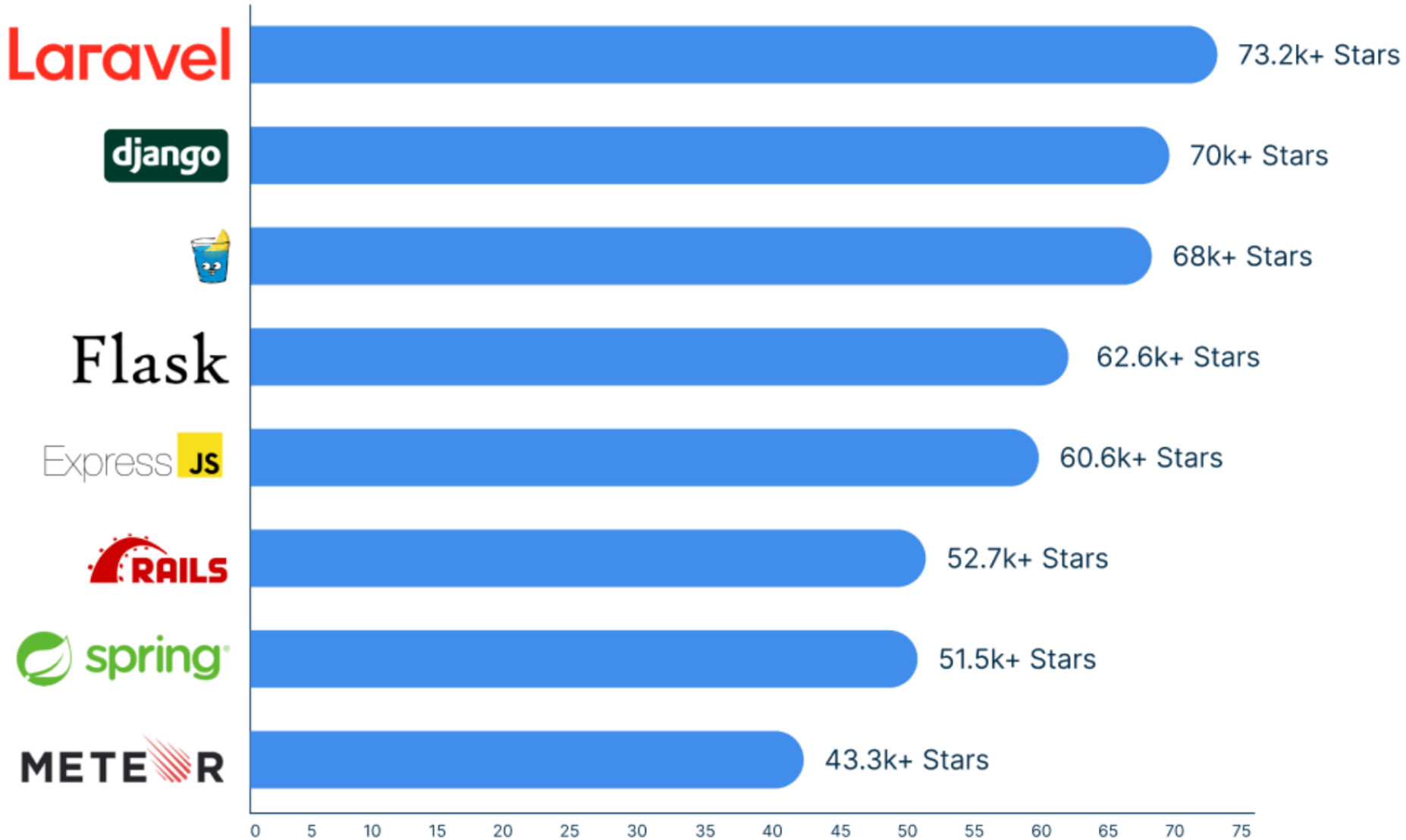
Backend

Focuses on building code, debugging, database management.

Programming languages:
Node.js, Python, Java

PLUS Php!!!!

6 Most Popular Backend Frameworks to Use in 2026



Choose Your Python Web Framework



Django

Batteries-included. Full-featured.
Everything you need.



FastAPI

Fast. Modern. Built for APIs.
High performance, async-first.



Flask

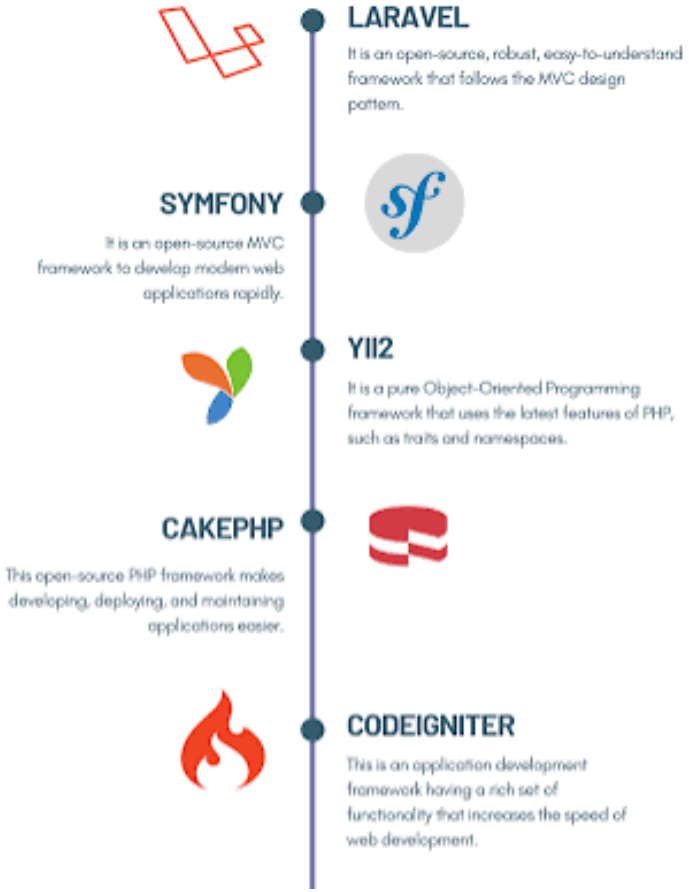
Lightweight. Flexible.
Simple and minimal.



https://micropyramid.com/blog/which-is-the-best-python-framework-for-web-development/

PHP Frameworks





What problems do backend web frameworks solve?

WHY USE A BACKEND FRAMEWORK?

Frameworks solve the hard parts around your code

01	**Security vulnerabilities**	Safer defaults for SQL, browser output and form requests
02	**Database integration**	ORMs and migrations replace scattered SQL and manual schema changes
03	**Routing and requests**	A routing table maps method + URL to the right handler
04	**Sessions and authentication**	Identity, state and permissions persist safely across requests
05	**Code structure and scale**	MVC and conventions keep a growing team oriented

LARAVEL

One opinionated “batteries included” backend stack

- Secure request and response defaults
- Eloquent ORM and migrations
- Routing, sessions, auth and MVC conventions

The framework handles repeatable infrastructure so your code can focus on the application's unique rules.

Secure defaults make common mistakes harder

SQL injection	Bound parameters keep user input separate from the SQL command.
Cross-site scripting (XSS)	Output escaping prevents user-supplied markup from executing in the browser.
Cross-site request forgery (CSRF)	A secret token proves that a state-changing form came from your application.

Frameworks reduce risk; they do not replace secure design, validation and review.

LARAVEL

Three protections you can point to

- Eloquent and the query builder use parameter binding
- Blade {{ }} escapes output by default
- @csrf plus middleware validates form tokens

```
{{ $name }}
@csrf
```

The safe path should be the easy path—and the dangerous path should be obvious.

Frameworks turn database access into application code

Object-Relational Mapping

Work with models, objects and collections in the application's language. The ORM translates those operations into SQL and maps rows back into objects.

Database migrations

Store schema changes as versioned files. Every developer and deployment can apply the same changes in the same order.

Model → **ORM** → **SQL database**

+ migrations track the schema

LARAVEL

Eloquent + migrations

- Models represent tables and relationships
- Query with PHP instead of assembling raw SQL
- Migration files make schema changes repeatable

```
Product::where(...)-
>get();
php artisan migrate
```

Raw SQL still exists—but it becomes a deliberate tool rather than the default structure of the application.

A routing table replaces chains of URL conditionals

METHOD	PATH	HANDLER
GET	/profile	ProfileController@show
POST	/profile	ProfileController@update
GET	/products/{id}	ProductController@show
DELETE	/products/{id}	ProductController@destroy

The framework also parses route parameters, query strings and form data—and returns a 404 when no route matches.

LARAVEL

Routes live in one readable table

- Method + URL map to a closure or controller
- {post} becomes a route parameter
- Request objects carry query and form data

```
Route::get('/profile', ...);
Route::delete('/posts/{post}',
...);
```

Clean URLs become an explicit interface instead of a growing nest of if statements and regular expressions.

Identity and state have to survive more than one request

Authentication

Register users, hash passwords, verify credentials and identify the current user.

Session management

Use a secure cookie or token to connect separate requests and store temporary state.

Authorization

Protect routes and actions with roles, permissions, policies or middleware.

Login answers “who are you?” Authorization answers “are you allowed to do this?”

LARAVEL

Services for web and API identity

- auth middleware protects routes
- Hash stores passwords; sessions keep state
- Policies/gates enforce permissions; Sanctum issues API tokens

```
->middleware('auth')
Hash::make($password)
```

A production-ready login system is infrastructure—not a weekend form and a password column.

Shared architecture prevents spaghetti code

Model

Data and business rules

View

Presentation and templates

Controller

Coordinates requests and responses

Separation of concerns

Each file has a clear job, so changes stay local and testing becomes easier.

Team collaboration

Standard folders and conventions tell every developer where to look and where new code belongs.

LARAVEL

A predictable project map

- routes/web.php defines entry points
- Controllers coordinate; Eloquent models hold data rules
- Blade renders output; Artisan generates files in the right folders

```
app/Http/Controllers/
app/Models/
resources/views/
```

Framework conventions trade some freedom for a codebase that remains navigable as features and people multiply.

Face-off Between Core PHP and Laravel

RADIX

Core PHP	vs	Laravel
A modular structure	**Structure**	An MVC structure
A faster development process	**Code reusability**	Beats this with code reusability
Enhanced code flexibility	**Flexibility**	Strict development rules
No caching mechanism	**Caching**	Facilitates cache back ends with multiple configurations
No external dependencies	**External dependency**	External dependencies exist
Needs security rules to be integrated during development	**Security**	Default authorization and authentication systems
No default data communication authorization	**Data communication**	Data communication is authorized with security token
Does not support default error and exception handling facilities	**Error and exception handling**	Error and exception handling protocols are already configured

CODE SUMMARY

Laravel adds a request lifecycle

The documented lifecycle is broader than MVC: middleware and the router dispatch to a route or controller, which returns a response.

CORE PHP — MANUAL WIRING

```
$id = (int) $_GET['id'];
$stmt = $pdo->prepare(
  'SELECT * FROM users WHERE id = ?'
);
$stmt->execute([$id]);
$user = $stmt->fetch();
echo json_encode($user);
```

LARAVEL — ROUTE → CONTROLLER → MODEL → VIEW

```
Route::get('/users/{user}',
  [UserController::class, 'show']);

public function show(User $user) {
  return view('user.profile',
    ['user' => $user]);
}
```

LIFECYCLE	public/index.php → bootstrap/app.php → HTTP kernel → providers → middleware/router → route/controller → response

Accel invests $57M into Laravel Products & Open-Source Framework

Laravel took a $57M Series A investment from Accel, Taylor Otwell continues to lead as CEO, & Laravel is doubling down on building powerful products for Web Artisans.

tl;dr - Laravel took a $57M Series A investment from Accel, Taylor Otwell continues to lead as CEO, & Laravel is doubling down on building powerful products for Web Artisans.





Share

Subs

Stay
your i

Em

LARAVEL HISTORY

15 years of open-source evolution

2011–2026

2011

Taylor Otwell releases Laravel as an open-source PHP framework.

OPEN SOURCE

The core remains publicly developed on GitHub under the permissive MIT license.

2024

Accel invests $57M; Taylor Otwell remains CEO and the open-source framework stays central.

2026

Laravel 13 continues the annual release cadence; the core repository has more than 42,000 commits.

15 YEARS • MIT LICENSE • 42K+ COMMITS

Sources: Laravel GitHub, release notes, and blog

https://laravel.com/blog/accel-invests-57m-into-laravel

What is the difference between a library and a framework?

THE ONE BIG IDEA

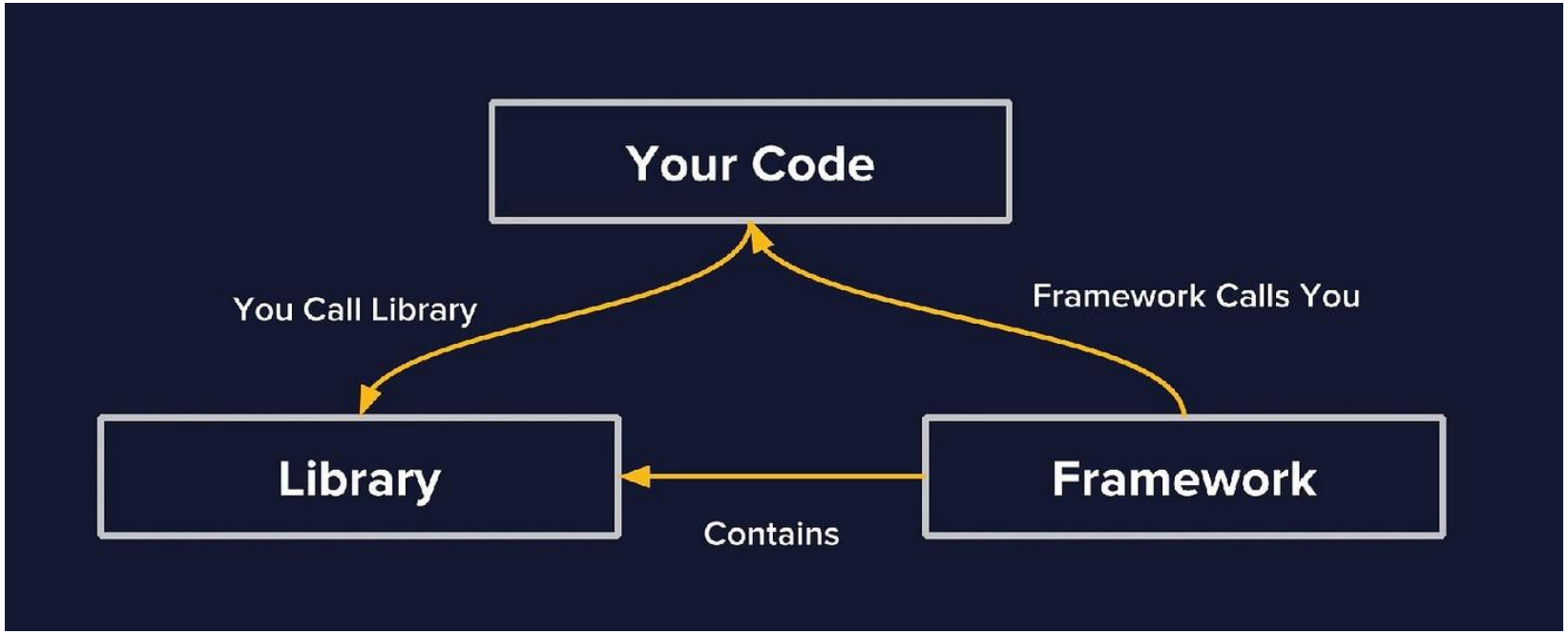
A library you call. A framework calls you.

A LIBRARY

You're in charge. You decide when to call it, and you could stop using it tomorrow.

A FRAMEWORK

It's in charge. It reads the request, decides which of your functions to run, and calls it.



https://medium.com/swlh/the-easiest-concept-between-library-and-framework-f7c6134dcc6c

INVERSION OF CONTROL

Dependency injection is one specific form of inversion of control.

IoC changes who directs execution; DI changes who supplies an object's collaborators.

THE GENERAL PRINCIPLE

INVERSION OF CONTROL

The framework owns the main flow and decides when to call application-defined hooks, handlers, or controllers.

Question: Who controls the sequence?

THE SPECIFIC WIRING PATTERN

DEPENDENCY INJECTION

A container or assembler provides dependencies from the outside, so the receiving class need not construct or locate them itself.

Question: Who supplies the collaborators?

CONTROL FLOW



DEPENDENCY FLOW



READ THE DEFINITIONS

Fowler paper — IoC Containers and Dependency Injection

Fowler bliki — Inversion of Control

Laravel docs — Service Container

IoC is the broader framework principle; DI is a mechanism for assembling dependencies.

DEPENDENCY INJECTION · MAILER EXAMPLE

The order service should ask for a mailer—not build one.

The business action places the order. Laravel's service container decides which mailer implementation to provide.

CREATED INSIDE THE SERVICE

```
final class PlaceOrder
{
  public function handle(Order $order)
  {
    $mailer = new SmtpMailer();
    $mailer->sendReceipt($order);
  }
}
```

PlaceOrder is now coupled to SMTP, constructs infrastructure itself, and is harder to test without sending mail.

INJECTED BY THE CONTAINER

```
interface OrderMailer {
  public function sendReceipt(Order $order);
}

final class PlaceOrder
{
  public function __construct(
    private OrderMailer $mailer
  ) {}

  public function handle(Order $order) {
    $this->mailer->sendReceipt($order);
  }
}
```

THE CONTAINER CONNECTS THE TWO

```
$this->app->bind(OrderMailer::class, SmtpMailer::class);
```

When PlaceOrder is resolved, inject SmtpMailer

RUNTIME FLOW

Controller → Service container → PlaceOrder → SmtpMailer

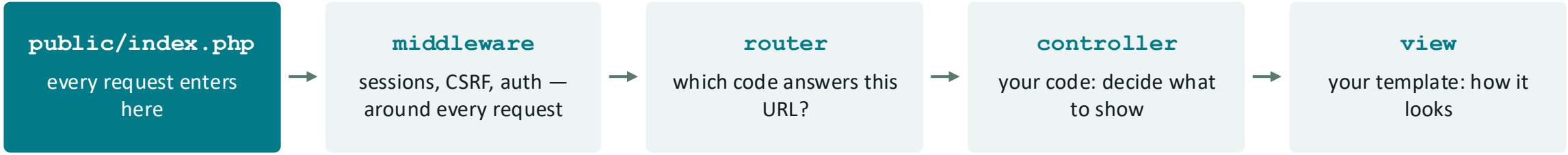
TESTING

Inject FakeOrderMailer in tests; use SmtpMailer in production. PlaceOrder stays unchanged.

LARAVEL REQUEST LIFECYCLE

ONE REQUEST, START TO FINISH

What happens between the URL and your code



- Laravel's public/index.php is 20 lines. Its last line hands the request to the framework: $app->handleRequest(Request::capture());
- You write the last two boxes. The framework owns the first three.
- PHP still starts fresh for every request — Laravel boots, answers, and is gone, exactly like week 3.

laravel.com/docs/13.x/lifecycle — the same journey, in the docs. Short, and worth reading once.

First Steps

The entry point for all requests to a Laravel application is the `public/index.php` file. All requests are directed to this file by your web server (Apache / Nginx) configuration. The `index.php` file doesn't contain much code. Rather, it is a starting point for loading the rest of the framework.

The `index.php` file loads the Composer generated autoloader definition, and then retrieves an instance of the Laravel application from `bootstrap/app.php`. The first action taken by Laravel itself is to create an instance of the application / service container.

```php
<?php

use Illuminate\Foundation\Application;
use Illuminate\Http\Request;

define('LARAVEL_START', microtime(true));

// Determine if the application is in maintenance mode...
if (file_exists($maintenance = __DIR__.'/../storage/framework/maintenance.php')) {
    require $maintenance;
}

// Register the Composer autoloader...
require __DIR__.'/../vendor/autoload.php';

// Bootstrap Laravel and handle the request...
/** @var Application $app */
$app = require_once __DIR__.'/../bootstrap/app.php';

$app->handleRequest(Request::capture());
```

PATTERN 1 · FRONT CONTROLLER

One door for every request

WEEK 3 — APACHE SENDS EVERYTHING TO index.php

```
# docker/apache.conf
FallbackResource /index.php

// public/index.php
require __DIR__ . '/../vendor/autoload.php';
$pantry = new Pantry(...);
// ...then 240 lines of routing,
// escaping and responding
```

LARAVEL — public/index.php, ALL OF IT THAT MATTERS

```
require __DIR__.'/../vendor/autoload.php';

$app = require_once
    __DIR__.'/../bootstrap/app.php';

$app->handleRequest(Request::capture());

// that's the whole file -- 20 lines
```

- Same pattern both times: one entry point, so every request gets the same setup, the same error handling, the same security.
- The difference is where the work went. In Laravel, index.php does almost nothing — it hands off.

Open inclass/wk7/laravel/public/index.php. It is the shortest file you'll read this week.

PATTERN 2 · ROUTING TABLE

Routes are Supported Paths/Methods

WEEK 3 — MATCH THE URL BY HAND

```
if ($method === 'GET'
    && $path === '/api/products') {
    respond(200, $pantry->list($today));
}
if ($method === 'GET' && preg_match(
    '#^/api/products/(\d+)$#', $path, $m)) {
    respond(200, $pantry->get((int) $m[1]));
}
```

LARAVEL — routes/web.php

```
Route::get('/hello', function () {
    return 'Hello, Laravel!';
});

Route::get('/products/{name}',
    fn (string $name) => "Product: $name");

Route::post('/products', ...);
```

- Method + URL → code. {name} in the URL arrives as $name — no regular expression.
- The framework does the matching, in order, and sends a 404 when nothing matches.
- php artisan route:list prints the table it built. Try it after drill 1.

Drills 1 and 2.

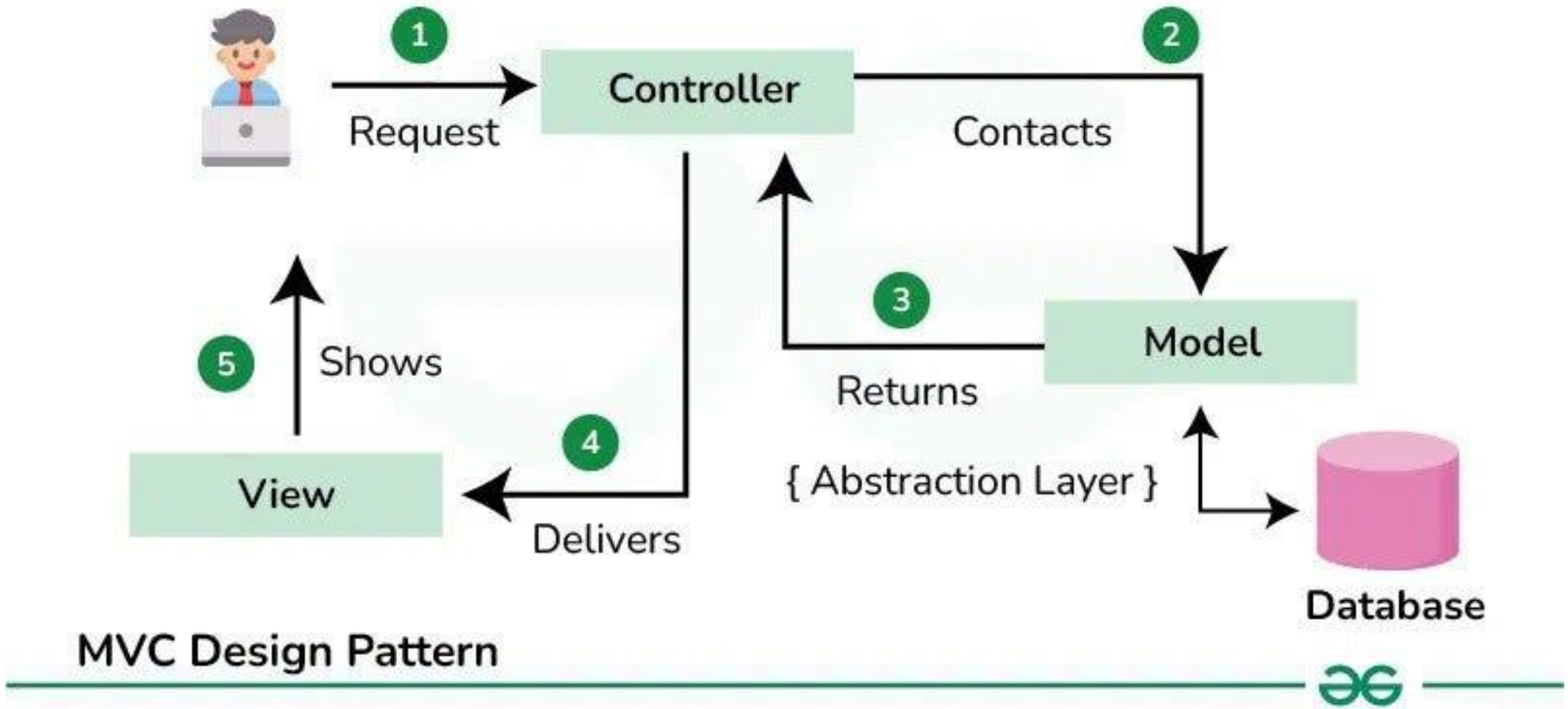
PATTERN 3 · MODEL, VIEW, CONTROLLER

Three jobs, three places

Model	the data and its rules	`app/Models/`	*Eloquent — week 9*
View	how it looks	`resources/views/*.blade.php`	*drills 3–5*
Controller	which data, which view	`app/Http/Controllers/`	*drills 6, 9*

Week 3 kept the rules in src/ and everything else in index.php. MVC splits “everything else” in two: what to show, and how it looks.

You've met the view half already: a React component is a view. Blade is the server-side version.



https://medium.com/@finzyphinzy/demystifying-mvc-understanding-the-model-view-controller-architecture-85c88a558951

MVC RESPONSIBILITIES

Each MVC layer has one job.

The controller coordinates the request without replacing the model or the view.

MODEL

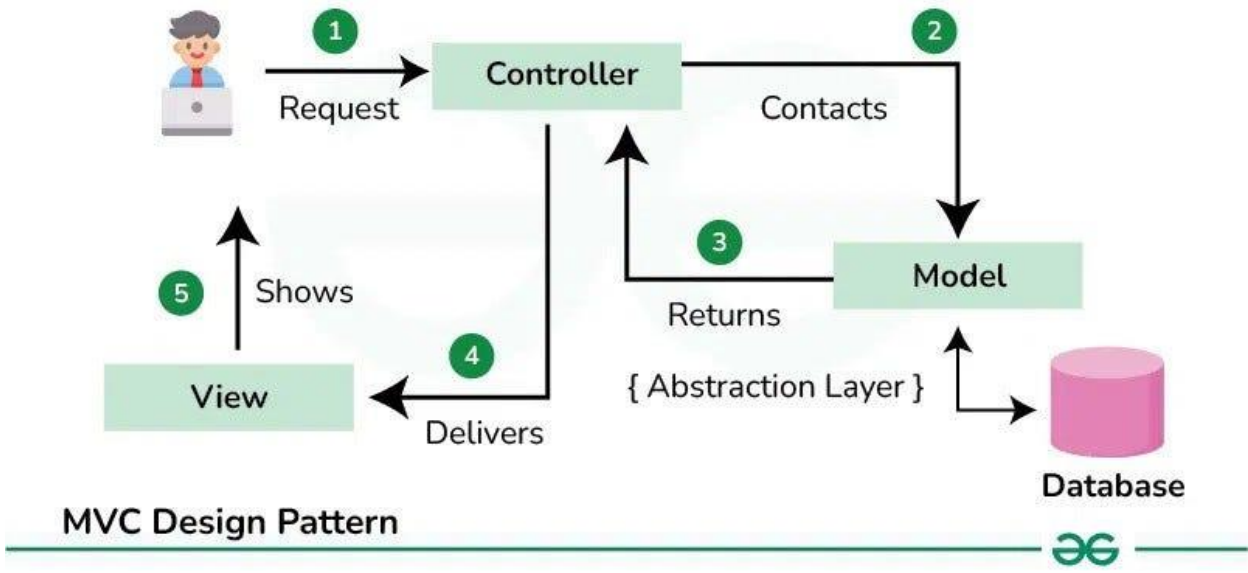
Holds application data and business logic. In Laravel, models are usually Eloquent classes that interact with the database.

VIEW

Provides the presentation layer—the Blade templates that render the data and interface shown to the user.

CONTROLLER

Receives the request, asks models or services for what is needed, and passes the results to a view or response.



In short: the model knows, the view shows, and the controller coordinates.

BACKGROUND · WHAT ESCAPING MEANS

Escaping makes untrusted input display as text—not run as code.

WITHOUT ESCAPING

```
User enters:
<script>alert('XSS')</script>

The browser sees real HTML tags.
<script> creates an element, and the JavaScript may run
inside your page.
```

AFTER ESCAPING

```
Template sends:
&lt;script&gt;alert(&#039;XSS&#039;)&lt;/script&gt;

The browser displays:
<script>alert('XSS')</script>

It is visible text—not executable markup.
```

- Escaping replaces special characters with HTML entities: < becomes <, > becomes >, and quotes are encoded too.
- The browser turns those entities back into visible characters, but does not reinterpret them as HTML or JavaScript.
- Escape when placing untrusted data into output. This is different from validating input or protecting a database query.

Next: Blade's {{ }} syntax escapes output by default; {!! !!} deliberately outputs raw HTML.

PATTERN 4 · TEMPLATES THAT ESCAPE

Safe unless you ask otherwise

WEEK 3 — ESCAPE EVERY VALUE, BY HAND

```
function e(mixed $s): string {
    return htmlspecialchars((string) $s,
                            ENT_QUOTES);
}

<td><?= e($p['name']) ?></td>
<td><?= $p['note'] ?></td>  // miss one: XSS
```

LARAVEL — BLADE

```
<td>{{ $product['name'] }}</td>
    {{-- escaped: a <script> shows as text --}}

<td>{!! $product['note'] !!}</td>
    {{-- NOT escaped: only for HTML you
         wrote yourself --}}
```

- Week 3 was safe only where someone remembered e(). One forgotten call is a cross-site scripting hole.
- Blade flips the default: {{ }} escapes, so forgetting makes you safe. The dangerous form, {!! !!}, looks dangerous.
- Only what goes through Blade is protected. A route that returns a plain string sends it as-is.

Drill 4: open /notes before you fix it.

PATTERN 5 · LAYOUTS

Write the page shell once

resources/views/components/layout.blade.php

```
@props(['title' => 'Pantry'])
<!doctype html>
<html>
<head><title>{{ $title }} · Pantry</title></head>
<body>
  <nav> ... </nav>
  <main>{{ $slot }}</main>
</body>
</html>
```

A PAGE THAT USES IT

```
<x-layout title="About">
    <h1>About this pantry</h1>
    <p>...</p>
</x-layout>

{{-- everything between the tags
     arrives in the layout as $slot --}}
```

$slot is React's children, on the server. Card didn't know what was in its footer (week 6, drill 4); the layout doesn't know what's on the page.

Drill 5. Every page in the workshop except /about already uses the layout.

PATTERN 6 · THE REQUEST AS AN EXPLICIT INPUT

Laravel hands the current request to your controller.

HIDDEN INPUT — REACH INTO GLOBAL STATE

```
public function index()
{
    $sort = $_GET['sort'] ?? null;
}

// The method has a hidden dependency:
// it only works when PHP globals exist.
```

EXPLICIT INPUT — DECLARE WHAT YOU NEED

```
public function index(Request $request)
{
    $sort = $request->query('sort');
}

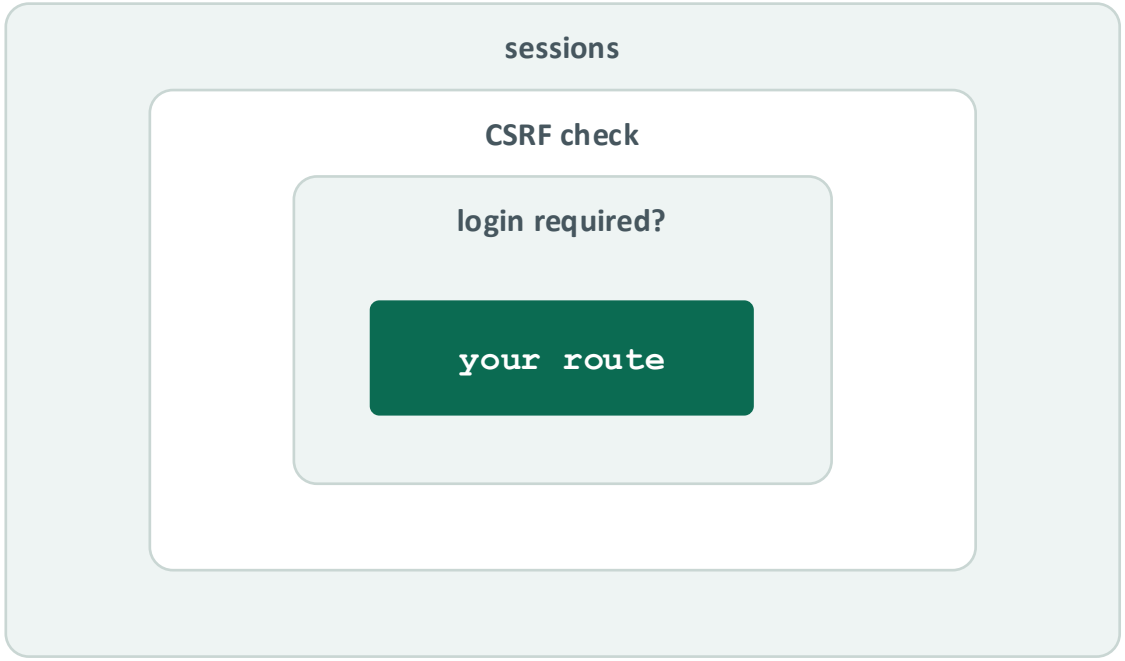
// Laravel calls index(...) and passes
// the Request object into this parameter.
```

- A browser request enters Laravel, and Laravel creates a Request object containing its URL, query string, form data, headers, and method.
- When Laravel prepares to call index(), it sees the Request type hint and supplies the current Request object.
- The controller no longer reaches into hidden global state. Its input is visible, easier to test, and consistent with the rest of the framework.

The core idea: you write the method and declare its needs; Laravel decides when to call it and provides those needs.

PATTERN 7 · MIDDLEWARE

Code that wraps every request



- A request passes through each layer on the way in, and the response on the way out.
- Laravel's web routes already run inside sessions, cookies and CSRF protection. You didn't write any of it.
- Week 10 (forms, sessions, logins) is mostly learning what these layers do — and adding your own.

Week 3 had none of these. Its forms could be submitted from any website — that's the CSRF hole.

PATTERN 8 · CONVENTION OVER CONFIGURATION

Someone already decided where things go

path	what lives there
`routes/web.php`	which URL runs which code
`app/Http/Controllers/`	the classes routes point at
`resources/views/`	Blade templates; components/ for shared pieces
`config/ and .env`	settings; .env differs per machine and is never committed
`tests/Feature/`	tests that make requests — the drill checks live here
`vendor/`	Laravel itself and ~100 packages (inside the container)

- Every Laravel app is laid out the same way. Open a stranger's project and you already know where the routes are.
- php artisan make:controller writes the file in the right place, with the right namespace.
- The name for this is “convention over configuration”: decide once, for everyone, so nobody decides every time.

Laravel took the idea, and the phrase, from Ruby on Rails.

WHAT YOU TRADE

Opinionated Web Frameworks

You get

- Routing, escaping, sessions, CSRF, error pages, a test harness — written, reviewed, maintained
- A layout every Laravel developer already knows
- Time back for the part only you can write

You give up

- Control: the framework decides when your code runs
- Visibility: more happens than you can see — “magic”, until you read how it works
- ~100 packages you didn't write, and upgrades on someone else's schedule

A framework is a bet that its authors' decisions are better than the ones you'd make under deadline. Usually it's a good bet. Know that you made it.

LARAVEL, THE PARTS YOU'LL TYPE TODAY

Routes and Blade on one slide

ROUTES — routes/web.php

```
Route::get('/hello', fn () => 'Hi');
Route::get('/products/{name}', fn ($name) => ...);
Route::get('/pantry', [ProductController::class,
                       'index'])
     ->name('pantry.index');
Route::post('/products', ...);

route('pantry.index')    // the URL, by name
abort(404);              // stop: Not Found
```

BLADE — resources/views/*.blade.php

```
{{ $item }}              {{-- escaped --}}

@foreach ($items as $item)
    <li>{{ $item }}</li>
@endforeach

@if ($items === []) <p>Empty.</p> @endif

<x-layout title="About"> ... </x-layout>
```

- A route returns a string, or view('name', [data]) to render resources/views/name.blade.php.
- Every @foreach needs an @endforeach. Forget one and the error mentions a different keyword.

RUNNING IT

Same Docker workflow as week 3

```
cd inclass/wk7/laravel
docker compose watch          # http://localhost:8000

# a second terminal:
docker compose exec app php artisan route:list
docker compose run --rm test    # every drill check

docker compose down            # stop
```

- No PHP or Composer on your laptop. The image has PHP 8.4, Composer, and Laravel 13 installed.
- Save a file and the watch terminal says Syncing. Reload the page — PHP reads your files again on every request.
- Check result on the drills page runs that drill's test. Friday, you'll read those tests — they're three lines each.

First build downloads PHP and the packages — a few minutes. If you haven't built it yet, start it now.

IN CLASS

Nine drills, one folder

Routes and views · 1–5

today

- 1 · a route is one line
- 2 · route parameters
- 3 · return a view
- 4 · escaping is the default
- 5 · a layout

Controllers and requests · 6–9

Friday

- 6 · a controller
- 7 · named routes
- 8 · a real 404
- 9 · the framework hands you the request

Hand-in: copy the laravel folder into inclass/wk7/ in your own repo and push it at the end of class — whatever you finished.

NINE DRILLS

Each one is something week 3 did by hand

#	drill	week 3, by hand	Laravel
1	**A route is one line**	if ($path === ...)	`Route::get('/hello', ...)`
2	**Route parameters**	preg_match and a capture group	`/products/{name}`
3	**Return a view**	HTML glued together in strings	`view('pantry', [...])`
4	**Escaping is the default**	htmlspecialchars() on every value	`{{ $note }}`
5	**A layout**	the page shell pasted into each page	`<x-layout> ... </x-layout>`
6	**A controller**	everything in one file	`[ProductController::class, 'index']`
7	**Named routes**	URLs typed into links	`route('pantry.index')`
8	**A real 404**	http_response_code(404)	`abort(404)`
9	**The request, handed to you**	$_GET, $_POST, $_SERVER	`index(Request $request)`

Edit the file each drill names, save, then press Check result on the drills page. It runs that drill's test.

DRILL 1 · ROUTES AND VIEWS

A route is one line

THE REQUEST

Add a route to routes/web.php: a GET request for /hello answers with the text Hello, Laravel!

WHAT IT DEMONSTRATES

Week 3 checked every URL with an if. Here each URL is one line in a table. The framework does the matching, and it calls your function.

IN THE DOCS

Routing — basic routing

WEEK 3 WOULD HAVE WRITTEN

```
if ($method === 'GET' && $path === '/hello') {
    echo 'Hello, Laravel!';
    exit;
}
```

LARAVEL — routes/web.php

```
Route::get('/hello', function () {
    return 'Hello, Laravel!';
});
```

Done when /hello says Hello, Laravel! and Check result says PASS.

Explain: who decides that /hello runs your function — you, or the framework?

DRILL 2 · ROUTES AND VIEWS

Route parameters

THE REQUEST

The route only answers /products/rice. Make it answer /products/ANYTHING, and say Product: followed by whatever came after the slash.

WHAT IT DEMONSTRATES

{name} in the URL arrives as $name in the function. Week 3 needed preg_match and a capture group for the same thing.

IN THE DOCS

Routing — route parameters

STARTER — THE BUG

```
Route::get('/products/rice', function () {
    return 'Product: rice';
});
```

THE FIX

```
Route::get('/products/{name}', function (string $name) {
    return "Product: $name";
});
```

Done when /products/Oats says Product: Oats, and /products/Rice says Product: Rice.

Explain: week 3 used preg_match('#^/api/products/(\d+)$#', ...) here. What does {name} save you?

DRILL 3 · ROUTES AND VIEWS

Return a view

THE REQUEST

The /pantry route builds HTML by gluing strings together. Move the markup into resources/views/pantry.blade.php and return the view.

WHAT IT DEMONSTRATES

The route decides what to show; the view decides how it looks. That split is the V in MVC — and a Blade view is the server-side cousin of a React component.

IN THE DOCS

Views, and Blade — loops

STARTER — THE BUG

```
$html = '<h1>Pantry</h1><ul>';
foreach ($items as $item) {
    $html .= '<li>'.$item.'</li>';
}
return $html.'</ul>';
```

THE FIX

```
return view('pantry', ['items' => $items]);

{{-- resources/views/pantry.blade.php --}}
<h1>Pantry</h1>
<ul> @foreach ($items as $item)
       <li>{{ $item }}</li> @endforeach </ul>
```

Done when /pantry shows the heading and three items, and the check confirms the pantry view made the page.

Explain: what does the route still decide, and what does the view decide now?

DRILL 4 · ROUTES AND VIEWS

Escaping is the default

THE REQUEST

Open /notes first. The alert is a <script> tag a stranger typed into a note, running in your page. In notes.blade.php, change {!! !!} to {{ }}.

WHAT IT DEMONSTRATES

{{ }} runs the value through htmlspecialchars() for you. Week 3 was safe only where someone remembered to. Blade is safe unless you ask otherwise.

IN THE DOCS

Blade — displaying data

STARTER — THE BUG

```
{{-- resources/views/notes.blade.php --}}
<p class="note">{!! $note !!}</p>

{{-- prints the note exactly as it arrived,
     <script> tag and all --}}
```

THE FIX

```
<p class="note">{{ $note }}</p>

{{-- escaped: the tag shows up as text --}}
```

Done when /notes shows the <script> tag as text, and nothing pops up.

Explain: in week 3's style, what happens the day someone forgets one htmlspecialchars()?

DRILL 5 · ROUTES AND VIEWS

A layout

THE REQUEST

/about has no nav bar and a bare tab title, because it isn't using the layout every other page shares. Wrap it in <x-layout title="About">.

WHAT IT DEMONSTRATES

The layout component prints whatever is between its tags as {{ $slot }}. One page shell, written once, for every page.

IN THE DOCS

Blade — components and slots

STARTER — THE BUG

```
{{-- resources/views/about.blade.php --}}
<h1>About this pantry</h1>
<p>A small Laravel app ...</p>
```

THE FIX

```
<x-layout title="About">
    <h1>About this pantry</h1>
    <p>A small Laravel app ...</p>
</x-layout>
```

Done when /about has the nav bar, and its tab title starts with About.

Explain: the layout prints what's between its tags as $slot. Which React idea from week 6 is that?

DRILL 6 · CONTROLLERS AND REQUESTS

A controller

THE REQUEST

Move the body of the /pantry route into ProductController's index() method, then replace the whole route with one line that points at it.

WHAT IT DEMONSTRATES

routes/web.php becomes a table of contents: which URL goes to which method. The code that answers lives in a class per kind of thing. The C in MVC.

IN THE DOCS

Controllers — basic controllers

BEFORE

```
Route::get('/pantry', function () {
    $items = ['Apples', 'Rice', 'Beans'];
    return view('pantry', ['items' => $items]);
});
```

AFTER

```
// routes/web.php
Route::get('/pantry', [ProductController::class, 'index']);

// app/Http/Controllers/ProductController.php
public function index() {
    $items = ['Apples', 'Rice', 'Beans'];
    return view('pantry', ['items' => $items]); }
```

Done when /pantry looks exactly the same, and the check confirms ProductController@index answered it.

Explain: routes/web.php is now a table of contents. Why is that worth an extra file?

DRILL 7 · CONTROLLERS AND REQUESTS

Named routes

THE REQUEST

Give the /pantry route a name, pantry.index. Then, in notes.blade.php, replace the hard-coded href="/pantry" with the route's name.

WHAT IT DEMONSTRATES

A link that asks for a route by name follows the route when its URL changes. A URL typed into a link breaks, silently, the day the route moves.

IN THE DOCS

Routing — named routes

BEFORE

```
Route::get('/pantry', [ProductController::class, 'index']);

<a href="/pantry">Back to the pantry</a>
```

AFTER

```
Route::get('/pantry', [ProductController::class, 'index'])
    ->name('pantry.index');

<a href="{{ route('pantry.index') }}">Back to the pantry</a>
```

Done when Check result says PASS.

Explain: change /pantry to /my-pantry in routes/web.php and click the link on /notes. What would the hard-coded link have done?

DRILL 8 · CONTROLLERS AND REQUESTS

A real 404

THE REQUEST

/items/99 answers “Not found” with status 200, which tells browsers and every API client that all is well. Make an unknown id a real 404.

WHAT IT DEMONSTRATES

Status codes are part of the answer. abort(404) stops the request right there, and Laravel sends a proper Not Found page with the right status.

IN THE DOCS

Error handling — HTTP exceptions

STARTER — THE BUG

```
Route::get('/items/{id}', function ($id) {
    $items = [1 => 'Apples', 2 => 'Rice', 3 => 'Beans'];
    return $items[$id] ?? 'Not found';   // status 200
});
```

THE FIX

```
Route::get('/items/{id}', function ($id) {
    $items = [1 => 'Apples', 2 => 'Rice', 3 => 'Beans'];
    abort_unless(isset($items[$id]), 404);
    return $items[$id];
});
```

Done when /items/2 says Rice. /items/99 shows Laravel's 404 page, and the Network tab says 404.

Explain: week 3 called http_response_code(404) and printed its own page. Where does Laravel's 404 page come from?

DRILL 9 · CONTROLLERS AND REQUESTS

The framework hands you the request

THE REQUEST

Do drill 6 first. /pantry?sort=az should list the items A to Z. Add Request $request to index() and read the query string from it.

WHAT IT DEMONSTRATES

You never call index() and never build a Request. Laravel reads the type, builds one, and passes it in: dependency injection. A test can build a Request. It can't fill $_GET.

IN THE DOCS

Requests — accessing the request

THE WEEK 3 WAY — WORKS IN THE BROWSER, FAILS THE CHECK

```
public function index() {
    $items = ['Apples', 'Rice', 'Beans'];
    if (($_GET['sort'] ?? '') === 'az') { sort($items); }
    return view('pantry', ['items' => $items]); }
```

THE FIX

```
public function index(Request $request) {
    $items = ['Apples', 'Rice', 'Beans'];
    if ($request->query('sort') === 'az') { sort($items); }
    return view('pantry', ['items' => $items]); }
```

Done when /pantry?sort=az lists Apples, Beans, Rice. Plain /pantry still lists Apples, Rice, Beans.

Explain: you never call index(), and you never build a Request. Who does both?

ITWS 2110 · LARAVEL · TROUBLESHOOTING

Laravel isn't working. Now what?

In the course's Docker container, and with Laravel Herd. The first question is always the same: where is PHP running?

The course supports Docker. Herd works for this week's drills, with the setup on the last slide.

FIRST QUESTION

Where is PHP running?

In the container (Docker)

- PHP, Composer, vendor/ and .env are inside the image. Your laptop has none of them.
- docker compose watch copies your edits in.
- Commands go through the container: docker compose exec app php artisan route:list
- The page is at http://localhost:8000

On your laptop (Herd)

- Herd installs PHP and Composer on the laptop itself.
- vendor/ and .env live in your folder, and you create them once.
- Commands run directly: php artisan route:list
- The page is wherever you served it: php artisan serve, or a Herd .test site.

Most “it doesn't work” is a command run in the wrong place, or two copies running and you're looking at the other one.

Pick one per folder. Before debugging anything else, say out loud which one you're using.

IN EITHER PLACE

Four things to look at before changing code

1	**Laravel's error page**	Read the first line. It names the problem and usually the file and line.	`(in the browser)`
2	**The routing table**	Is your URL in it? Is it pointing where you think?	`php artisan route:list`
3	**The log**	Errors the page didn't show you, newest last.	`storage/logs/laravel.log`
4	**What Laravel thinks it is**	Version, environment, debug mode, URL.	`php artisan about`

IN DOCKER, PUT THIS IN FRONT OF EVERY php artisan COMMAND

```
docker compose exec app php artisan route:list
docker compose exec app tail -n 30 storage/logs/laravel.log
```

Stale page after a fix? php artisan optimize:clear empties Laravel's caches, including compiled views.

IN THE CONTAINER

What goes wrong with Docker

what you see	why	do this
The first docker compose watch takes minutes	It is downloading PHP and about a hundred packages. Once.	`wait; later starts take seconds`
port is already allocated	Something else is using port 8000	`APP_PORT=8001 docker compose watch`
Your edit doesn't show up	Watch isn't running. Each save should print Syncing	`docker compose watch`
You changed composer.json, phpunit.xml or .env.example and nothing happened	Those are built into the image, not synced	`docker compose build`
php: command not found, or vendor/autoload.php: Failed to open stream	You ran artisan on your laptop. PHP and vendor/ are in the container	`docker compose exec app php artisan ...`
You want the server's own output	It goes to the container's log	`docker compose logs -f app`

docker compose down stops it. Nothing in this workshop is lost when the container goes: your files are on your laptop.

BEFORE THE HERD PROBLEMS

What is Laravel Herd?

A free Mac and Windows app from the Laravel team. It puts PHP, Composer and a web server on your laptop — the things our Docker image carries.

What it installs

- PHP, and a switch between versions
- Composer and the laravel installer
- A web server (nginx) and Node
- Its own commands: herd link, herd open, herd log

How it serves a site

- A folder inside ~/Herd is served by itself, as foldername.test
- A folder anywhere else: Add Site in the app, or herd link
- Or skip the site and run php artisan serve

What it doesn't do

- Create vendor/ or .env for a project you cloned. You run composer install yourself
- Databases: MySQL and the other services are in the paid Herd Pro
- Run on Linux

herd.laravel.com · Optional in this course. Docker is what the homework instructions assume.

WITH HERD

What goes wrong on the laptop

what you see	why	do this
Failed to open stream … vendor/autoload.php	No vendor/ folder: the packages were never installed	`composer install`
500 Server Error; the log says: No application encryption key has been specified.	No .env file, or no APP_KEY in it	`cp .env.example .env` `php artisan key:generate`
Failed to listen on 127.0.0.1:8000 (reason: Address already in use)	The Docker copy, or another serve, already has that port	`docker compose down` `or  php artisan serve --port=8001`
php artisan test prints JSON instead of the usual report	Laravel's dev tools detected an AI coding tool's terminal (Claude Code, Cursor…)	`run it in an ordinary terminal`
Odd errors right after install	Laravel 13 needs PHP 8.3 or newer	`php -v` `herd php:list`

Herd's own tools: herd restart · herd log · herd link · herd open · herd which-php

SAME FOLDER, TWO WAYS TO RUN IT

The whole setup, side by side

DOCKER — WHAT THE COURSE SUPPORTS

```
cd inclass/wk7/laravel
docker compose watch
# http://localhost:8000

docker compose run --rm test
docker compose exec app php artisan route:list
docker compose down
```

HERD — PHP ON YOUR LAPTOP

```
cd inclass/wk7/laravel
composer install
cp .env.example .env
php artisan key:generate
php artisan serve     # http://127.0.0.1:8000
# or Add Site in Herd:  http://laravel.test
php artisan test
php artisan route:list
```

- Both run the same drills, and Check result works in both — under php artisan serve and on a Herd .test site.
- Don't run both at once on port 8000, and don't commit vendor/ or .env — the .gitignore already leaves them out.
- From week 9 the app needs MySQL. The course's Compose files provide it; with Herd you would supply your own.